Containers, Pods, Kubernetes

https://upstreamwithoutapaddle.com/

References: Core Kubernetes, Kubernetes in Action (Manning Press)

Topics

- Why Kubernetes exists (and overview)
- Why the Pod
- Build a Pod
- CGroups for Pod processes

Future Topics

- CNI and Pod networking
- Pod Storage
- Running Pods: how kubelet works
- DNS in Kubernetes

1. Why Kubernetes Summary

- Why Kubernetes exists
- Key terms commonly used in K8S world
- Infrastructure Drift problem
- Containers and Images
- Core foundation of Kubernetes
- Kubernetes features
- Kubernetes components and architecture
- Summary

Why Kubernetes Exists

- Kubernetes is open-source platform for hosting containers
- Defines APIs for provisioning storage, networking, security to support applications
- Continuously reconciles state of deployments
- Provides stable environment for development and deployment of software

Key Terms 1/2

- OCI – Open Container Initiative image format for executable self-contained applications
- Container – OCI image that runs an application
- Pod – Kubernetes object encapsulating running containers
- Deployment – collection of pods managed by Kubernetes
- Kubectl – command line tool for talking to control plane (oc is enhanced version)

Key Terms 2/2

- Kubelet – Kubernetes agent that runs on cluster nodes to implement control-plane commands
- Node – machine that runs a kubelet process
- DaemonSet – like deployment that runs on all cluster nodes
- CNI – container networking interface (for pods)
- CSI – container storage interface (for pods)

Infrastructure Drift Problem

- Infrastructures configurations change over time
  - Java version, location of applications, changing hardware, undocumented infrastructure changes
- Causes applications not to work
- Kubernetes manages state using REST API and declarative state
  - using kubectl, oc, or programatic calls to API
- Kubernetes manages both applications and services in containers (microservices)
  - standardizing on containers reduces drift problem

Containers and Images

- Apps have dependencies to run
- Containers images package these dependencies
- Docker was one way to run containers
  - satisfying OCI (OCI Image Spec)
  - containerd cri-o and runc are other container runtimes
- Images contain multiple tar files, each is a layer
  - layers can contain Linux binaries and app files
- Container runtime unpacks images and runs it on host kernel

Containers Aid Isolation

- Containers packaging required software to run app
- Also allow different apps to have separate libraries
  - or versions
- Simplifies running software across multiple servers
- To run multiple containers, automation needed
  - Kubernetes
  - Handles deployment, scaling, fault tolerance, authentication, services and routers, etc.

Text Files are Foundation of K8S

- Everything defined by text files (YAML or JSON)
  - running OCI images, configuring network rules, role-based authentication and authorization (RBAC), etc.
- Suppose have Docker file to build MySQL image:

```
FROM alpine:3.7
RUN apk add --no-cache mysql-client
ENTRYPOINT ["mysql"]
```

- Build image (docker build) and push to registry (docker push) that is visible to Kubernetes cluster
  - such as docker hub or RedHat quay.io

Text Files Continued Example

- Next build and push container to talk to DB:

```
FROM python:3.7
WORKDIR /myapp
COPY src/requirements.txt ./
RUN pip install -r requirements.txt
COPY src /myapp
CMD [ "python", "mysql-custom-
client.py" ]
```

- To run both containers as 2 pods, create YAML file and run using kubectl (or oc)
  - oc create -f my-app.yaml
  - see next slide

Example my-app.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: core-k8s
  spec:
  containers:
    - name: my-mysql-server
      image: myregistry.com/mysql-server:v1.0
---
apiVersion: v1
kind: Pod
metadata:
  name: core-k8s
  spec:
  containers:
    - name: my-sqlclient
      image: myregistry.com/mysql-custom-client:v1.0
      command: [‘tail’,'-f',’/dev/null’]
```

What happens in Example?

- `oc` or `kubectl` reads file and calls API on K8S which stores the contents of the YAML file
- Based on the YAML file, 2 pods are created with the images pulled from the repository
- Kubernetes will monitor that the pods were created and retry if there are problems
- Kubernetes schedules the pods to run on available hosts and monitors all application state
- Improves over classical servers and VMs

All Infrastructure Rules in YAML

- Configuration of ports or IP routes
- Persistent storage availability
- Hosting software on specific servers
- Security provisioning, network rules for access
- DNS configuration per-application and global
- YAML (or JSON) files provide desired status
  - Control loops ensure *actual* becomes *desired*

Why Pods and Kubernetes Need Each Other

- Expose cloud-neutral API
- Integrate with all major cloud and hypervisor platforms
- Fault-tolerant framework (via etcd database)
- Manage deployments with minimal downtime
- Automate scaling with rolling updates
- Create integrations with load balancing
- High availability platform
- Allow service discovery via DNS

More Kubernetes Features

- Run batch processes (known as Jobs)
- Include API extensions using Custom Resource Definitions (CRD)
- Enable inspection of failed processes cluster-wide
- Enable remote execution into any container (via oc exec and oc describe tools)
- Allow mounting of local and remote storage to a container with declarative storage volumes
- Allows upgrades of cluster and apps without downtime

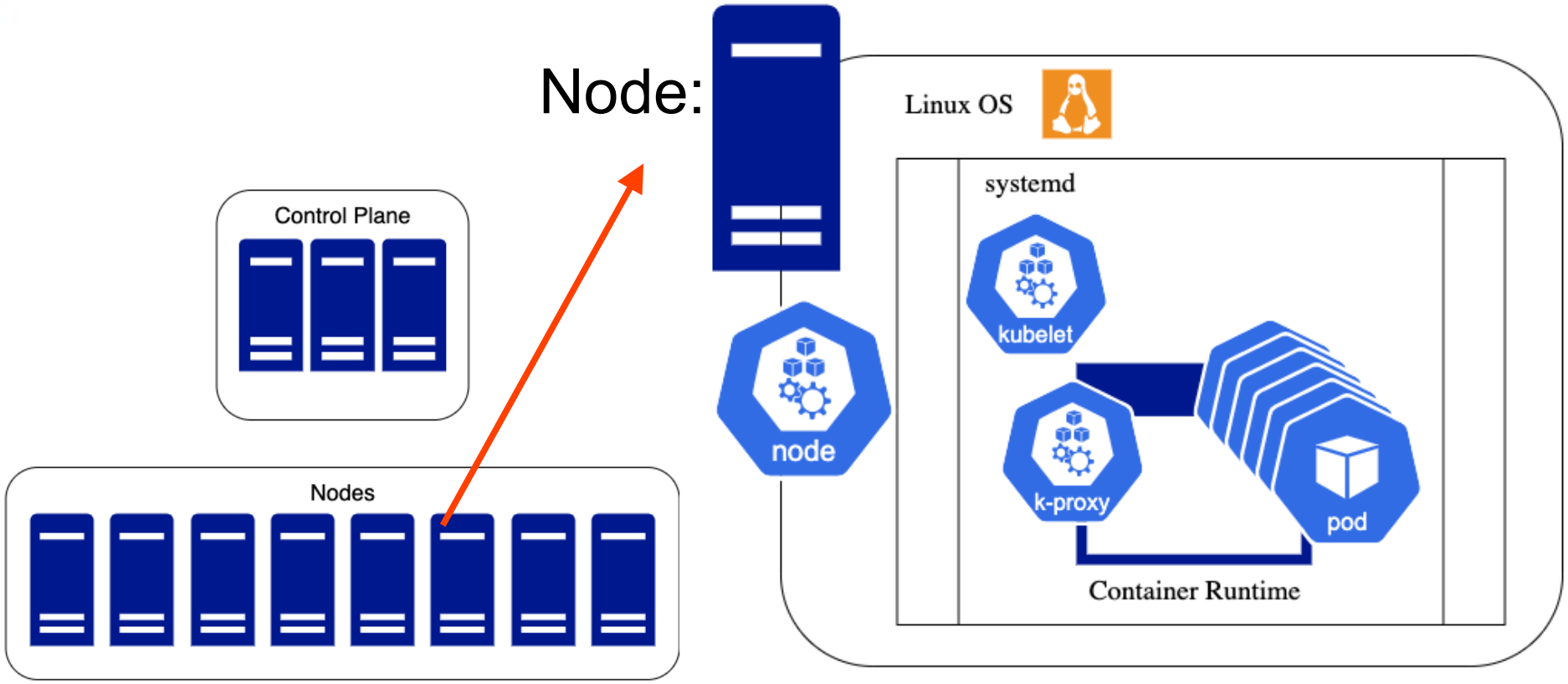
Response to HW Failure

- Node stops responding to the control plane
- Control plane schedules Pods to other Nodes
- API server provides information on failed node and new Pod locations
- Clients communicating to pods' services are rerouted to new locations
- Attached storage connected to Pods on old node are moved to new Pod locations

Where to Run Kubernetes

- For fault tolerant versions need a couple of servers (ours have 3 TiB RAM and 232 cores)
- For development several options
  - minikube, kind are popular
  - Also microshift, K3d, K3s, MicroK8s (have not tried)
    - microshift runs on Raspberry Pi4 Model B
- Most take significant RAM (need >= 16GiB in total)

K8S Components and Architecture



(OpenShift is Kubernetes)

- Node agent, kubelet
- CNI (Container Network Interface)
- CRI (Container Runtime Interface)

Kubernetes Cluster Contains

- Computers (real or virtual)
- Network infrastructure
  - switches, routers, IP management, DNS servers, load balancers
- Storage infrastructure
  - disks, NAS, NFS endpoints
- Container registry
  - place to host images

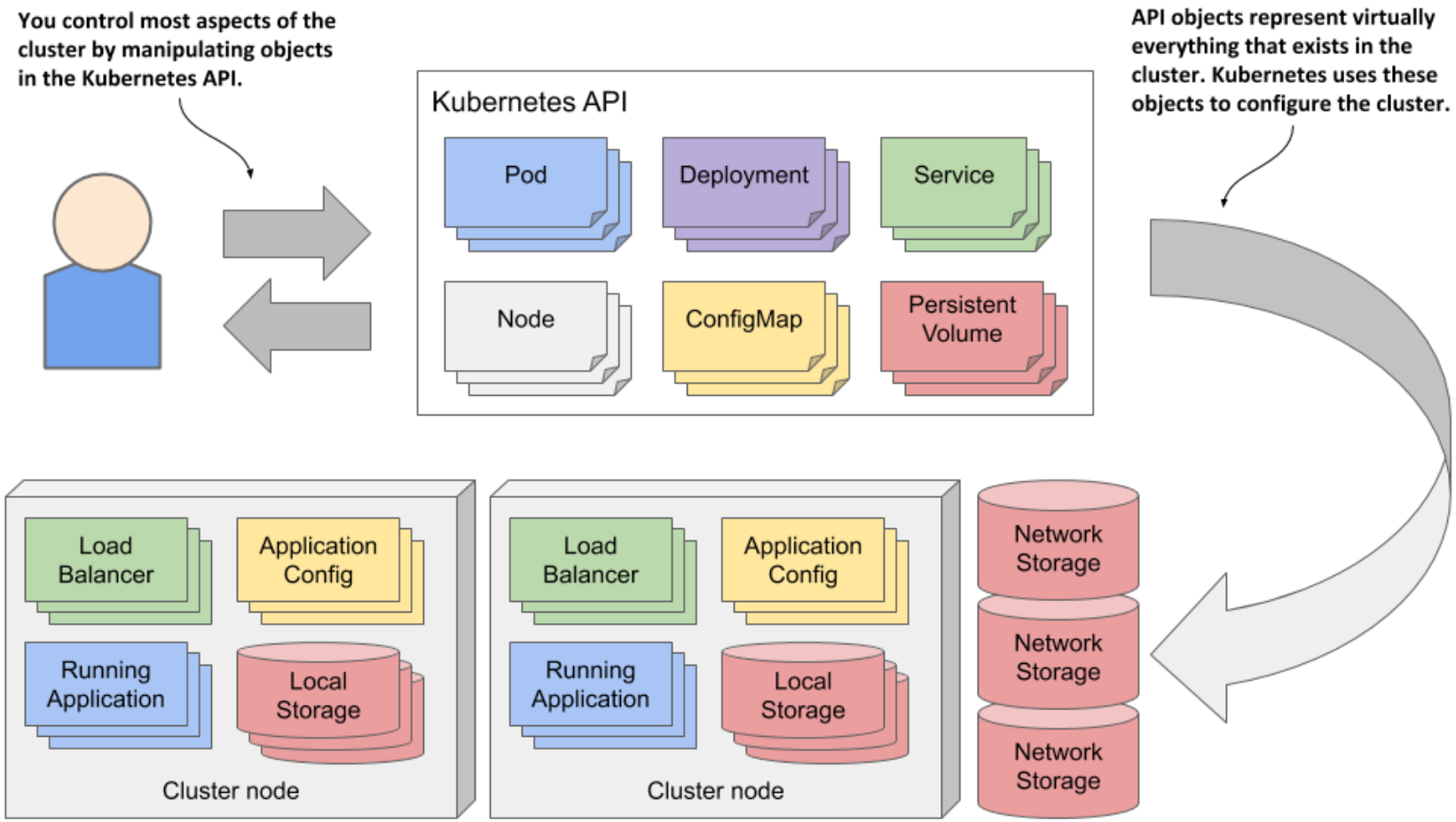
Kubernetes Components

- Nodes (3 to hundreds, sometimes thousands)
- Scheduler
  - kube-scheduler assigns pods to nodes
- API server
  - kube-apiserver a RESTful server that handles all requests for something to happen
- Controller manager
  - kube-controller-manager runs control loops to keep system consistent with desired state

Kubernetes API

- Administering a K8S platform: declare API objects
  - Kubernetes does the rest
- kube-apiserver allows CRUD of all API objects and provides REST interface
- API objects have
  - a version, named `apiVersion`
  - kind, such as `kind: Deployment`, and
  - a metadata section
- run `oc api-resources` to get a list

Kubernetes API



From Kubernetes in Action 2nd Edn MEAP, by Marco Luksa, 2021

Important API Objects

- Pods and Deployments
- Kubernetes standard services
- Ingress services
- PersistentVolumes and PersistentVolumeClaims
- NetworkPolicies
- CustomResources
- API Objects may be associated with a namespace (equals OKD Project) or not

Introduction Summary

- Kubernetes makes development easier
- Kubernetes builds an ecosystem of components to enable robust scalable systems
- Everything can be done with `kubectl` (or `oc`)
- Kubernetes is a cluster of computers to deploy containers
  - with container orchestration, storage management and distributed networking
- The Pod is the building block of Kubernetes